UNIVERSITÀ DI TRENTO
Department of
Information Engineering and Computer Science

Automated Reasoning and Formal Verification

Laboratory 1

Gabriele Masina
gabriele.masina@unitn.it

Università di Trento

March 04, 2026

These slides are derived from those by Giuseppe Spallitta for FM lab 2021-2024.

Course overview

When? Every **Wednesday, 11:30-13:30**

Course overview

When? Every **Wednesday, 11:30-13:30**

What? Labs: **model and solve** logic problems (SAT, SMT, Model Checking)

Course overview

When? Every **Wednesday, 11:30-13:30**

What? Labs: **model and solve** logic problems (SAT, SMT, Model Checking)

How? Examples in class + optional homeworks (highly recommended)

Course overview

When? Every **Wednesday, 11:30-13:30**

What? Labs: **model and solve** logic problems (SAT, SMT, Model Checking)

How? Examples in class + optional homeworks (highly recommended)

Slides and other material:

`https://github.com/masinag/arfv2026`

What to expect from the labs

What labs will be about

- ✓ **Hands-on** experience with SAT/SMT solvers

What to expect from the labs

What labs will be about

- ✓ **Hands-on** experience with SAT/SMT solvers
- ✓ **Modelling** of problems in SAT and SMT and usage of advanced features

What to expect from the labs

What labs will be about

- ✓ **Hands-on** experience with SAT/SMT solvers
- ✓ **Modelling** of problems in SAT and SMT and usage of advanced features

. . . and in the second module

What to expect from the labs

What labs will be about

- ✓ **Hands-on** experience with SAT/SMT solvers
- ✓ **Modelling** of problems in SAT and SMT and usage of advanced features

. . . and in the second module

- ✓ **Model checking** of discrete and timed systems

What to expect from the labs

What labs will be about

- ✓ **Hands-on** experience with SAT/SMT solvers
- ✓ **Modelling** of problems in SAT and SMT and usage of advanced features

. . . and in the second module

- ✓ **Model checking** of discrete and timed systems

What labs will not be about

What to expect from the labs

What labs will be about

- ✓ **Hands-on** experience with SAT/SMT solvers
- ✓ **Modelling** of problems in SAT and SMT and usage of advanced features

. . . and in the second module

- ✓ **Model checking** of discrete and timed systems

What labs will not be about

- ✗ **Algorithms** for SAT/SMT solving or model checking

What to expect from the exams

The exam will contain exercises similar to the ones shown during lectures

- AR only: one SAT exercise and one SMT exercise.
- AR&FV: one SAT/SMT exercise and one Model Checking exercise.

Outline

1. First steps on SAT solving
2. Getting used with DIMACS format
3. Simple real-life applications

Why SAT solving?

SAT solving

Is a Boolean formula satisfiable?

$$(A \vee B) \wedge (\neg A \vee C) \wedge (\neg B \vee C) \wedge C$$

Why SAT solving?

SAT solving

Is a Boolean formula satisfiable?

$$(A \vee B) \wedge (\neg A \vee C) \wedge (\neg B \vee C) \wedge C$$

Answer: YES, for example with $A = \top, B = \bot, C = \top$.

Why SAT solving?

SAT solving

Is a Boolean formula satisfiable?

$$(A \vee B) \wedge (\neg A \vee C) \wedge (\neg B \vee C) \wedge C$$

Answer: YES, for example with $A = \top, B = \bot, C = \top$.

But where do these problems come from?

Why SAT solving?

SAT solving

Is a Boolean formula satisfiable?

$$(A \vee B) \wedge (\neg A \vee C) \wedge (\neg B \vee C) \wedge C$$

Answer: YES, for example with $A = \top, B = \bot, C = \top$.

But where do these problems come from?

- Software and hardware verification

Why SAT solving?

SAT solving

Is a Boolean formula satisfiable?

$$(A \vee B) \wedge (\neg A \vee C) \wedge (\neg B \vee C) \wedge C$$

Answer: YES, for example with $A = \top, B = \bot, C = \top$.

But where do these problems come from?

- Software and hardware verification
- Planning and scheduling

Why SAT solving?

SAT solving

Is a Boolean formula satisfiable?

$$(A \vee B) \wedge (\neg A \vee C) \wedge (\neg B \vee C) \wedge C$$

Answer: YES, for example with $A = \top, B = \bot, C = \top$.

But where do these problems come from?

- Software and hardware verification
- Planning and scheduling
- Cryptography

Why SAT solving?

SAT solving

Is a Boolean formula satisfiable?

$$(A \vee B) \wedge (\neg A \vee C) \wedge (\neg B \vee C) \wedge C$$

Answer: YES, for example with $A = \top, B = \bot, C = \top$.

But where do these problems come from?

- Software and hardware verification
- Planning and scheduling
- Cryptography
- **Logic puzzles!**

How to solve SAT problems?

In these labs, we will:

- learn how to model problems

How to solve SAT problems?

In these labs, we will:

- learn how to model problems
- use efficient **SAT solvers** to solve them

How to solve SAT problems?

In these labs, we will:

- learn how to model problems
- use efficient **SAT solvers** to solve them

As a solver we will use **MathSAT 5** (`https://mathsat.fbk.eu/download.html`):

How to solve SAT problems?

In these labs, we will:

- learn how to model problems
- use efficient **SAT solvers** to solve them

As a solver we will use **MathSAT 5** (`https://mathsat.fbk.eu/download.html`):

- Efficient SAT and SMT (spoiler...) solver developed by FBK and UniTN

How to solve SAT problems?

In these labs, we will:

- learn how to model problems
- use efficient **SAT solvers** to solve them

As a solver we will use **MathSAT 5** (`https://mathsat.fbk.eu/download.html`):

- Efficient SAT and SMT (spoiler...) solver developed by FBK and UniTN
- It support basic SAT operators and algorithms

How to solve SAT problems?

In these labs, we will:

- learn how to model problems
- use efficient **SAT solvers** to solve them

As a solver we will use **MathSAT 5** (`https://mathsat.fbk.eu/download.html`):

- Efficient SAT and SMT (spoiler...) solver developed by FBK and UniTN
- It support basic SAT operators and algorithms
- It also supports a wide range of SMT theories (including equality and uninterpreted functions, linear arithmetic, bit-vectors, and arrays)

Other SAT solvers

Several open-source SAT solvers:

- Minisat `https://github.com/niklasso/minisat`
- Glucose `https://github.com/audemard/glucose`
- CaDiCaL `https://github.com/arminbiere/cadical`

Annual "SAT competition" is held to improve SAT solvers:

`https://satcompetition.github.io/2026/tracks.html`

Isn't SAT solving NP-complete?

Solving SAT is hard, but SAT solvers are very efficient in many cases

Isn't SAT solving NP-complete?

Solving SAT is hard, but SAT solvers are very efficient in many cases

Modelling the problem in the right way can make a **huge difference**!

The DIMACS format

How do we interact with SAT solvers?

The DIMACS format

```
c quinn.cnf
c
p cnf 7 8
 1   2 0
-2  -4 0
 3   4 0
-4  -5 0
 5  -6 0
 6  -7 0
 6   7 0
 7  -3 0
```

How do we interact with SAT solvers?

The standard input format accepted by SAT solvers is called **DIMACS format**

It is a simple **text format** to describe CNF formulas

The DIMACS format: an example

```
c quinn.cnf
c
p cnf 7 8
 1   2 0
-2  -4 0
 3   4 0
-4  -5 0
 5  -6 0
 6  -7 0
 6   7 0
 7  -3 0
```

Comments

Each row starting with a **lower case** c **is a comment**

$\Rightarrow$ used to explain the encoding and other useful information

The DIMACS format: an example (cont.d)

```
c quinn.cnf
c
p cnf 7 8
 1   2 0
-2  -4 0
 3   4 0
-4  -5 0
 5  -6 0
 6  -7 0
 6   7 0
 7  -3 0
```

Problem line

The first non-comment line must be the problem line, starting with a lower case p

- The first word is the **problem type** (in our case CNF)
- The first number is the **number of variables**
- The second number is the **number of clauses**

The DIMACS format: an example (cont.d)

```
c quinn.cnf
c
p cnf 7 8
 1   2 0
-2  -4 0
 3   4 0
-4  -5 0
 5  -6 0
 6  -7 0
 6   7 0
 7  -3 0
```

Problem line

The first non-comment line must be the problem line, starting with a lower case p

- The first word is the **problem type** (in our case CNF)
- The first number is the **number of variables**
- The second number is the **number of clauses**

Notice

The formula must be in CNF:

$$(l_{11} \vee l_{12} \vee \dots) \wedge (l_{21} \vee l_{22} \vee \dots) \wedge \dots$$

Remember: any formula can be converted in CNF in linear time

The DIMACS format: an example (cont.d)

```
c quinn.cnf
c
p cnf 7 8
 1   2 0
-2  -4 0
 3   4 0
-4  -5 0
 5  -6 0
 6  -7 0
 6   7 0
 7  -3 0
```

Clauses

Each subsequent row contains a single clause:

- A clause (disjunction of literals) is represented as a **sequence of integers**, where each integer represents a literal
- A *positive* integer i represents the *positive* literal x_i, a *negative* integer $-i$ represents the *negative* literal $\neg x_i$
- Each clause is **terminated by a 0**

The DIMACS format: an example (cont.d)

```
c quinn.cnf
c
p cnf 7 8
 1   2 0
-2  -4 0
 3   4 0
-4  -5 0
 5  -6 0
 6  -7 0
 6   7 0
 7  -3 0
```

Clauses

Each subsequent row contains a single clause:

- A clause (disjunction of literals) is represented as a **sequence of integers**, where each integer represents a literal
- A *positive* integer i represents the *positive* literal x_i, a *negative* integer $-i$ represents the *negative* literal $\neg x_i$
- Each clause is **terminated by a 0**

The first three clauses are:

- $(\ x_1 \vee \ x_2)$
- $(\neg x_2 \vee \neg x_4)$
- $(\ x_3 \vee \ x_4)$

MathSAT output

Once you create your file using the DIMACS format, you can feed it to the solver:

- If there is a solution, the solver returns SAT. We can ask the solver to return a **model** (i.e. a satisfying assignment) for the formula

MathSAT output

Once you create your file using the DIMACS format, you can feed it to the solver:

- If there is a solution, the solver returns SAT. We can ask the solver to return a **model** (i.e. a satisfying assignment) for the formula
- If no solution exists to the problem, the solver returns UNSAT

MathSAT output

Once you create your file using the DIMACS format, you can feed it to the solver:

- If there is a solution, the solver returns SAT. We can ask the solver to return a **model** (i.e. a satisfying assignment) for the formula
- If no solution exists to the problem, the solver returns UNSAT

```
./mathsat -input=dimacs -model quinn.cnf
s SATISFIABLE
v -1 2 3 -4 5 6 7 0
```

Outline

1. First steps on SAT solving
2. Getting used with DIMACS format
3. Simple real-life applications

First encodings

Exercise 1.1

Encode the following Boolean formulas and check if they are satisfiable:

- $\varphi_1 := (x_1 \vee \neg x_5 \vee x_4) \wedge (\neg x_3 \vee x_4) \wedge (\neg x_1 \vee x_5 \vee x_2)$
- $\varphi_2 := (x_1 \rightarrow x_2) \vee x_3$

First encodings

Exercise 1.1

Encode the following Boolean formulas and check if they are satisfiable:

- $\varphi_1 := (x_1 \vee \neg x_5 \vee x_4) \wedge (\neg x_3 \vee x_4) \wedge (\neg x_1 \vee x_5 \vee x_2)$
- $\varphi_2 := (x_1 \rightarrow x_2) \vee x_3$

- The first formula can be easily encoded using the DIMACS format!

First encodings

Exercise 1.1

Encode the following Boolean formulas and check if they are satisfiable:

- $\varphi_1 := (x_1 \vee \neg x_5 \vee x_4) \wedge (\neg x_3 \vee x_4) \wedge (\neg x_1 \vee x_5 \vee x_2)$
- $\varphi_2 := (x_1 \rightarrow x_2) \vee x_3$

- The first formula can be easily encoded using the DIMACS format!
- The second formula must be written into CNF format before feeding it to the solver!

$$\varphi_2 = (\neg x_1 \vee x_2) \vee x_3$$

First encodings (cont.d)

Homework 1.1

Encode the following Boolean formulas and check if they are satisfiable:

- $\varphi_3 := \neg x_1 \rightarrow (x_1 \rightarrow x_2)$
- $\varphi_4 := (x_1 \leftrightarrow x_2)$

Outline

1. First steps on SAT solving

2. Getting used with DIMACS format

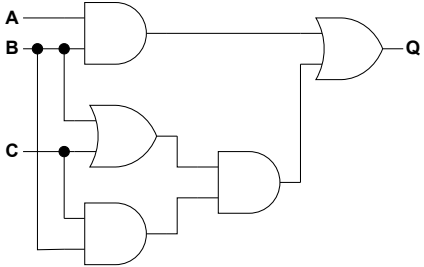
3. Simple real-life applications
   - Testing Boolean circuits
   - From DIMACS to SMT-LIB
   - Solving simple logic puzzles

Testing Boolean circuits

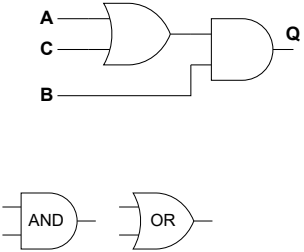
Exercise 1.2

You are asked to build a circuit for a top-secret project. Each gate costs a lot of money, so you suggest an alternative and cheaper circuit. Are the two circuits are equivalent?

Original circuit:

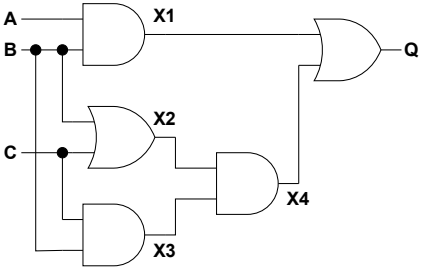


Simplified circuit:



Testing Boolean circuits (cont.d)

The first step is encoding the two Boolean circuit. Starting with the first one, let's define intermediate variables to store the result of each inner gate and the final output:



Testing Boolean circuits (cont.d)

Each gate represents a clause of the final encoding. The upper left AND gate can be represented as:

$$X1 \leftrightarrow A \wedge B$$

If we use DIMACS, this formula must be converted into its equivalent CNF form. In this case:

$$\begin{aligned}
&(X1 \rightarrow (A \wedge B)) \wedge (X1 \leftarrow (A \wedge B)) \\
\equiv &(\neg X1 \vee (A \wedge B)) \wedge (\neg(A \wedge B) \vee X1) \\
\equiv &(\neg X1 \vee A) \wedge (\neg X1 \vee B) \wedge (\neg A \vee \neg B \vee X1)
\end{aligned}$$

The same pattern can be applied for each AND gate!

Testing Boolean circuits (cont.d)

The leftmost OR gate can be represented as:

$$X2 \leftrightarrow B \vee C$$

This formula must be converted into its equivalent CNF form:

$$\begin{aligned}
&(X2 \rightarrow (B \vee C)) \wedge (X2 \leftarrow (B \vee C)) \\
\equiv &(\neg X2 \vee (B \vee C)) \wedge (\neg(B \vee C) \vee X2) \\
\equiv &(\neg X2 \vee B \vee C) \wedge ((\neg B \wedge \neg C) \vee X2) \\
\equiv &(\neg X2 \vee B \vee C) \wedge (\neg B \vee X2) \wedge (\neg C \vee X2)
\end{aligned}$$

The same pattern can be applied for each OR gate!

Testing Boolean circuits (cont.d)

- Using the two retrieved formulas, we can encode both circuits into a CNF equivalent formula.
- We are missing the last step: checking the equivalence of the two systems!
  - If at least one counter-example is found (the two output are not identical for the same input), then the two circuits are not equivalent.
  - As a consequence, we can encode this condition into the sub-formula:

$$\neg(O1 \leftrightarrow O2)$$

Satisfiability corresponds to the desired counter-example, thus the non-equivalence.

Testing Boolean circuits (cont.d)

As always we must convert into into CNF form:

$$
\begin{aligned}
&\neg((O1 \to O2) \wedge (O2 \to O1)) \\
\equiv &\neg((\neg O1 \vee O2) \wedge (\neg O2 \vee O1)) \\
\equiv &((O1 \wedge \neg O2) \vee (O2 \wedge \neg O1)) \\
\equiv &(O1 \vee O2) \wedge (O1 \vee \neg O1) \wedge (\neg O2 \vee O2) \wedge (\neg O2 \vee \neg O1)
\end{aligned}
$$

Each clause in the form $A \vee \neg A$ is always true, so they are useless and can be removed:

$$(O1 \vee O2) \wedge (\neg O2 \vee \neg O1)$$

Testing Boolean circuits (cont.d)

- We can now pass the encoding to the solver... after we map each variable into indexed integers!

Testing Boolean circuits (cont.d)

- We can now pass the encoding to the solver... after we map each variable into indexed integers!
- The solver returns UNSAT
  $\Rightarrow$ no counter-example has been found
  $\Rightarrow$ the two circuits are equivalent!

Using DIMACS could be problematic...

In Exercise 1.2, we had to rewrite a non-CNF formula to feed it to the SAT solver.

What happens if the problem to encode is drastically bigger?

- Solution 1: use scripts to map variables into their index and apply CNF-ization...
  $\Longrightarrow$ boring and error-prone :(

Using DIMACS could be problematic...

In Exercise 1.2, we had to rewrite a non-CNF formula to feed it to the SAT solver.

What happens if the problem to encode is drastically bigger?

- Solution 1: use scripts to map variables into their index and apply CNF-ization...
  $\Longrightarrow$ boring and error-prone :(
- Solution 2: use SMT-LIB, another format most SMT solvers accept!

MathSAT input format: SMT-LIB

- SMT-LIB is an international initiative aimed at facilitating research and development in SMT, the language is only one of several goals achieved
- All details about SMT-LIB can be found here: `https://smtlib.cs.uiowa.edu/`
- SMT-LIB is more powerful than you could expect. We will discover its potential during the course

General structure of a SMT-LIB file

A typical SMT-LIB file is characterized by the following sections:

- The **Options** section
- The **Declarations** section
- The **Assertions** section
- The **Commands** section

SMT-LIB file: Options

The header can contain some commands to enable **additional functionalities**, e.g.:

- Generation of models $\Longrightarrow$ `(set-option :produce-models true)`

SMT-LIB file: Options

The header can contain some commands to enable **additional functionalities**, e.g.:

- Generation of models $\Longrightarrow$ `(set-option :produce-models true)`
- Extraction of UNSAT cores

SMT-LIB file: Options

The header can contain some commands to enable **additional functionalities**, e.g.:

- Generation of models $\Longrightarrow$ `(set-option :produce-models true)`
- Extraction of UNSAT cores
- Extraction of interpolants

SMT-LIB file: Options

The header can contain some commands to enable **additional functionalities**, e.g.:

- Generation of models $\Longrightarrow$ `(set-option :produce-models true)`
- Extraction of UNSAT cores
- Extraction of interpolants
- Set background logic for more efficient computations

SMT-LIB file: Options

The header can contain some commands to enable **additional functionalities**, e.g.:

- Generation of models $\Longrightarrow$ `(set-option :produce-models true)`
- Extraction of UNSAT cores
- Extraction of interpolants
- Set background logic for more efficient computations

We will highlight the most popular options and their effects with examples

SMT-LIB file: Options

The header can contain some commands to enable **additional functionalities**, e.g.:

- Generation of models $\Longrightarrow$ `(set-option :produce-models true)`
- Extraction of UNSAT cores
- Extraction of interpolants
- Set background logic for more efficient computations

We will highlight the most popular options and their effects with examples

Warning

Each command must be written inside parentheses (...)!

SMT-LIB file: Declarations

In this section we must declare variables and functions used to describe the problem

Variables can be declared with the following syntax:

```
(declare-const <name> <type>)
```

SMT-LIB file: Declarations

In this section we must declare variables and functions used to describe the problem

Variables can be declared with the following syntax:

```
(declare-const <name> <type>)
```

Types supported by SMT-LIB are:

SMT-LIB file: Declarations

In this section we must declare variables and functions used to describe the problem

Variables can be declared with the following syntax:

```
(declare-const <name> <type>)
```

Types supported by SMT-LIB are:

- `Bool` *(we will use only this for now)*

SMT-LIB file: Declarations

In this section we must declare variables and functions used to describe the problem

Variables can be declared with the following syntax:

```
(declare-const <name> <type>)
```

Types supported by SMT-LIB are:

- `Bool` *(we will use only this for now)*
- `Int`

SMT-LIB file: Declarations

In this section we must declare variables and functions used to describe the problem

Variables can be declared with the following syntax:

```
(declare-const <name> <type>)
```

Types supported by SMT-LIB are:

- `Bool` *(we will use only this for now)*
- `Int`
- `Real`

SMT-LIB file: Declarations

In this section we must declare variables and functions used to describe the problem

Variables can be declared with the following syntax:

```
(declare-const <name> <type>)
```

Types supported by SMT-LIB are:

- `Bool` *(we will use only this for now)*
- `Int`
- `Real`
- `(_ BitVec <size>)`

SMT-LIB file: Declarations

In this section we must declare variables and functions used to describe the problem

Variables can be declared with the following syntax:

```
(declare-const <name> <type>)
```

Types supported by SMT-LIB are:

- `Bool` *(we will use only this for now)*
- `Int`
- `Real`
- `(_ BitVec <size>)`
- `(Array <type> <type>)`

SMT-LIB file: Declarations

In this section we must declare variables and functions used to describe the problem

Variables can be declared with the following syntax:

```
(declare-const <name> <type>)
```

Types supported by SMT-LIB are:

- `Bool` *(we will use only this for now)*
- `Int`
- `Real`
- `(_ BitVec <size>)`
- `(Array <type> <type>)`

Functions can be declared with the following syntax:

```
(declare-fun <name> ([input types]) <type>)
```

SMT-LIB file: Assertions

- Next, we need to encode the constraints in the form of **assertions**:

```
(assert <condition>)
```

SMT-LIB file: Assertions

- Next, we need to encode the constraints in the form of **assertions**:
  `(assert <condition>)`
- Conditions can be basic $(x \vee y)$ or **nested** $((x \wedge y) \vee (\neg x \wedge y))$.

SMT-LIB file: Assertions

- Next, we need to encode the constraints in the form of **assertions**:
  `(assert <condition>)`
- Conditions can be basic $(x \vee y)$ or **nested** $((x \wedge y) \vee (\neg x \wedge y))$.

Warning

In SMT-LIB operators always use a prefix notation!

SMT-LIB Assertions: Propositional Logic

Boolean operators are available to use:

- NEGATION is represented as `(not <formula>)`

SMT-LIB Assertions: Propositional Logic

Boolean operators are available to use:

- NEGATION is represented as `(not <formula>)`
- OR is represented as `(or <formula1> <formula2>)`

SMT-LIB Assertions: Propositional Logic

Boolean operators are available to use:

- NEGATION is represented as `(not <formula>)`
- OR is represented as `(or <formula1> <formula2>)`
- AND is represented as `(and <formula1> <formula2>)`

SMT-LIB Assertions: Propositional Logic

Boolean operators are available to use:

- NEGATION is represented as `(not <formula>)`
- OR is represented as `(or <formula1> <formula2>)`
- AND is represented as `(and <formula1> <formula2>)`
- IMPLIES is represented as `(=> <formula1> <formula2>)`.

SMT-LIB Assertions: Propositional Logic

Boolean operators are available to use:

- NEGATION is represented as `(not <formula>)`
- OR is represented as `(or <formula1> <formula2>)`
- AND is represented as `(and <formula1> <formula2>)`
- IMPLIES is represented as `(=> <formula1> <formula2>)`.
- XOR can be represented as `(xor <formula1> <formula2>)`

SMT-LIB Assertions: Propositional Logic

Boolean operators are available to use:

- NEGATION is represented as `(not <formula>)`
- OR is represented as `(or <formula1> <formula2>)`
- AND is represented as `(and <formula1> <formula2>)`
- IMPLIES is represented as `(=> <formula1> <formula2>)`.
- XOR can be represented as `(xor <formula1> <formula2>)`
- IFF is represented as `(= <formula1> <formula2>)`

SMT-LIB Assertions: Propositional Logic

Boolean operators are available to use:

- NEGATION is represented as `(not <formula>)`
- OR is represented as `(or <formula1> <formula2>)`
- AND is represented as `(and <formula1> <formula2>)`
- IMPLIES is represented as `(=> <formula1> <formula2>)`.
- XOR can be represented as `(xor <formula1> <formula2>)`
- IFF is represented as `(= <formula1> <formula2>)`

Warning

The `and` and `or` operators can be have more than two arguments!

SMT-LIB file: Commands

The bottom part of the file describes the task the solver has to perform:

- First you should check satisfiability of the actual problem:

```
(check-sat)
```

SMT-LIB file: Commands

The bottom part of the file describes the task the solver has to perform:

- First you should check satisfiability of the actual problem:

```
(check-sat)
```

- We can then ask for a satisfying assignment:

```
(get-model)
```

SMT-LIB file: Commands

The bottom part of the file describes the task the solver has to perform:

- First you should check satisfiability of the actual problem:

```
(check-sat)
```

- We can then ask for a satisfying assignment:

```
(get-model)
```

- Lastly we end the file using:

```
(exit)
```

SMT-LIB file: Commands

The bottom part of the file describes the task the solver has to perform:

- First you should check satisfiability of the actual problem:

```
(check-sat)
```

- We can then ask for a satisfying assignment:

```
(get-model)
```

- Lastly we end the file using:

```
(exit)
```

Then we can run MathSAT as:

```
./mathsat quinn.smt2
```

Testing Boolean circuits using SMT-LIB

Exercise 1.2a

Try to solve Exercise 1.2 using SMT-LIB instead of DIMACS.

Testing Boolean circuits using SMT-LIB

Exercise 1.2a

Try to solve Exercise 1.2 using SMT-LIB instead of DIMACS.

Much easier, right? :)

The Importance of Modelling

Up to now we simply converted Boolean formulas in DIMACS/SMT-LIB format...

The Importance of Modelling

Up to now we simply converted Boolean formulas in DIMACS/SMT-LIB format...

... but real life is not so easy :)

The Importance of Modelling

Up to now we simply converted Boolean formulas in DIMACS/SMT-LIB format...

... but real life is not so easy :)

Modelling

The hardest part will be **modelling** the problem:

- Identify the variables to use
- Identify the constraints to encode

Sorting People

Exercise 1.3

Consider three chairs in a row and three guests: A, B e C. We know that:

- A does not want to sit next to C.
- A does not want to sit on the leftmost chair.
- B does not want to sit immediately to the right of C

Is it possible to satisfy the following constraints and find a valid placement?

Sorting people: Variables

First let's define the Boolean variables necessary to model the problem:

- x_{ij} states if user i ($i \in \{A,B,C\}$) sits in chair j ($j \in 1,2,3$)
- In this case $3 \cdot 3 = 9$ variables are needed

Sorting people: Properties (1)

Now let's encode the conditions stated in the text of the problem:

- A does not want to sit next to C

$$\neg(x_{A1} \wedge x_{C2}) \wedge \neg(x_{C1} \wedge x_{A2}) \wedge \neg(x_{A2} \wedge x_{C3}) \wedge \neg(x_{C2} \wedge x_{A3})$$

- A does not want to sit on the leftmost chair.

$$\neg x_{A1}$$

- B does not want to sit immediately to the right of C

$$\neg(x_{B2} \wedge x_{C1}) \wedge \neg(x_{B3} \wedge x_{C2})$$

Sorting people: Properties (2)

- Is this enough to model the problem?
- There are some "not obvious" conditions that must be provided!

Sorting people: Properties - Cardinality Constraints (3)

- Guest A must sit in at least one chair:

$$x_{A1} \vee x_{A2} \vee x_{A3}$$

The same applies for guests B and C, so we must add two additional conditions using the same pattern:

$$x_{B1} \vee x_{B2} \vee x_{B3}$$

$$x_{C1} \vee x_{C2} \vee x_{C3}$$

Sorting people: Properties - Cardinality Constraints (4)

- Guest A must sit in at most one chair:

$$x_{A1} \rightarrow (\neg x_{A2} \wedge \neg x_{A3})$$

$$x_{A2} \rightarrow (\neg x_{A1} \wedge \neg x_{A3})$$

$$x_{A3} \rightarrow (\neg x_{A1} \wedge \neg x_{A2})$$

- Guest B must sit in at most one chair
- Guest C must sit in at most one chair

The last two conditions can be encoded similarly to the first one, thus we encode them again simply changing the involved variables.

Sorting people: Properties - Cardinality Constraints (5)

- Only one person can sit on the first position:

$$x_{A1} \rightarrow (\neg x_{B1} \wedge \neg x_{C1})$$

$$x_{B1} \rightarrow (\neg x_{A1} \wedge \neg x_{C1})$$

$$x_{C1} \rightarrow (\neg x_{A1} \wedge \neg x_{B1})$$

The structure is identical to the previous formulas, thus we can repeat the same pattern changing the variables!
We need also to encode the same typology of clauses for the second and the third position.

Sorting people: Results

Now we can fed the encoding into a SAT solver
$\Rightarrow$ The solver returns **UNSAT**

Cracking Codes

Exercise 1.4

You have to guess a two digits code, with digits from 1 to 4. You have three hints:

- In 12, one number is correct and well placed.
- In 14, nothing is correct.
- In 43, one number is correct but wrongly placed.

Does a solution exist? **Is it unique**?

Cracking Codes: Variables

First let's define the Boolean variables necessary to model the problem:

- x_{ij} states if number i is placed in position j, with $i \in \{1, 2, 3, 4\}, j \in \{1, 2\}$.
- In this case $4 \cdot 2 = 8$ variables are needed

Cracking Codes: Properties (1)

Let's encode the first condition: "In 12, one number is correct and well placed".

- This means that either the digit 1 is correct and 2 is not part of the code or viceversa.

$$(x_{11} \wedge \neg x_{21} \wedge \neg x_{22}) \vee (x_{22} \wedge \neg x_{11} \wedge \neg x_{12})$$

Cracking Codes: Properties (2)

The second condition, "In 14, nothing is correct", is easier to encode:

- 1 and 4 must be excluded from the possible valid values in each position

$$\neg x_{11} \wedge \neg x_{12} \wedge \neg x_{41} \wedge \neg x_{42}$$

Cracking Codes: Properties (3)

Let's encode the third condition, "In 43, one number is correct but wrongly placed".

- Either the digit 3 should be put in position 1 and 4 is not part of the code or the digit 4 should be put in position 2 and 3 is not part of the code.

$$(x_{31} \wedge \neg x_{41} \wedge \neg x_{42}) \vee (x_{42} \wedge \neg x_{31} \wedge \neg x_{32})$$

Cracking Codes: Properties (4)

Similarly to the previous exercise, we must add some "hidden" conditions:

- Each position must contain at least a digit

$$(x_{11} \vee x_{21} \vee x_{31} \vee x_{41}) \wedge (x_{12} \vee x_{22} \vee x_{32} \vee x_{42})$$

- Each position must contain at most a digit.

$$x_{11} \rightarrow (\neg x_{21} \wedge \neg x_{31} \wedge \neg x_{41})$$

  This condition must be replicated for each digit, for both position 1 and 2. A total of 8 different formulas must be encoded!

Cracking Codes: Results

Once we map the variables into the usual indexed integers we can feed it to the SAT solver and see if there is a valid solution.

Cracking Codes: Results (cont.d)

Once we map the variables into the usual indexed integers we can feed it to the SAT solver and see if there is a valid solution.
$\Rightarrow$ The solver returns **SAT** and, checking the output, the two true variables generate 32 as solution.

Cracking Codes: Results (cont.d)

Once we map the variables into the usual indexed integers we can feed it to the SAT solver and see if there is a valid solution.
$\Rightarrow$ The solver returns **SAT** and, checking the output, the two true variables generate 32 as solution.

To check the uniqueness of this solution, we must ensure the two digits cannot be respectively 3 and 2. This can be easily encoded using the following clauses:

$$\neg x_{31} \vee \neg x_{22}$$

Cracking Codes: Results (cont.d)

Once we map the variables into the usual indexed integers we can feed it to the SAT solver and see if there is a valid solution.
$\Rightarrow$ The solver returns **SAT** and, checking the output, the two true variables generate 32 as solution.

To check the uniqueness of this solution, we must ensure the two digits cannot be respectively 3 and 2. This can be easily encoded using the following clauses:

$$\neg x_{31} \vee \neg x_{22}$$

$\Rightarrow$ Now the solver returns **UNSAT**, proving the uniqueness of the solution

Homeworks

Homework 1.3: Cheaters

Three students A, B and C are accused of having illegally obtained the questions for the Automated Reasoning exam. During the investigation process the students made the following statements:

- A said: "B is guilty and C is innocent"
- B said: "If A is guilty, then C is also guilty"
- C said: "I'm innocent and one of the others, perhaps even the two, are guilty"

Considering that all the students spoke the truth, which of the students are guilty and which are innocent? Solve it using a SAT solver.

Homeworks

Homework 1.4: Password

Using the digits 1,2,3 and 4 you need to create a 3-length password. There are some rules that must be fulfilled:

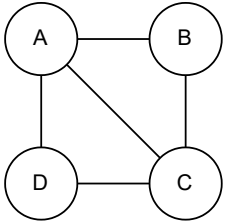
- The password should be even
- We cannot use the same digit three times, otherwise it would be easy to guess it.
- It is possible to repeat the same digit twice, just make sure the two digits are not adjacent.

Solve it using a SAT solver and report the solution. Is this unique?

Homeworks

Homework 1.5: Coloring Graph

You are given the graph shown in the figure on the right. Suppose you want to color the nodes of this graph so that nodes connected by an edge cannot have the same color. Given these assumptions:

- Is it possible to color the graph using only 2 colors?
- Is it possible to color the graph using only 3 colors?

Solve it using a SAT solver.